

FortifAI System Architecture Analysis & Recommendations

Executive Summary

After analyzing your FortifAI cybersecurity system, I've identified **critical privacy concerns** and architectural discrepancies from what you expected. This document provides:

1. **Current System Architecture** - How it actually works
 2. **Privacy Issues** - Why client data is visible in admin GUI
 3. **Federated Learning Analysis** - Mathematical explanation and convergence mechanics
 4. **Dataset Recommendations** - Public datasets for proper ML training
 5. **Architectural Recommendations** - How to achieve true privacy-enhanced federated learning
-

1. CURRENT SYSTEM ARCHITECTURE

1.1 What You Expected

- **Privacy-enhanced federated learning** where:
 - Clients train models locally on their own data
 - Only model parameters/gradients sent to server
 - Server aggregates models without seeing raw client data
 - Anomalies detected locally, only alerts sent to server

1.2 What Actually Happens

Critical Privacy Issue: Raw Data Transmission

The system currently sends **ALL RAW CLIENT DATA** to the server, violating federated learning principles:

Client Side (client_agentAI.py lines 5875-5978):

```
python
```

```
def send_telemetry(self):
    """Send telemetry data to server"""
    telemetry = {
        'client_id': CLIENT_ID,
        'timestamp': datetime.now().isoformat(),
        'network': self.network_data,    # ← RAW NETWORK DATA
        'processes': self.process_data,  # ← RAW PROCESS DATA
        'files': self.file_data,         # ← RAW FILE DATA
        'user_activity': self.user_data, # ← RAW USER DATA
        'anomalies': list(self.anomaly_detector.anomaly_alerts),
        'system_info': self.system_info
    }
    # Sends everything to server
    self.send_to_server('telemetry', telemetry)
```

Server Side (`admin_serverAI.py` lines 695-849):

```
python

def handle_telemetry(self, client_id, data):
    """Store ALL client telemetry in database"""
    # Stores raw network, process, file, user data
    cursor.execute("""
        INSERT INTO telemetry (client_id, timestamp, network_data,
                               process_data, file_data, user_activity, ...)
        VALUES (%s, %s, %s, %s, %s, %s, ...)
        """, (client_id, timestamp, json.dumps(data['network']), ...))
```

Why Admin Can See Everything: The admin GUI directly queries this database and displays all raw client data in tables and charts. This is **NOT privacy-preserving**.

2. FEDERATED LEARNING: CURRENT IMPLEMENTATION

2.1 How FL Currently Works

Client Side Process:

1. **Data Collection** (`collection_loop()` - lines 5826-5917)
 - Collects network, process, file, user activity data every 30 seconds
 - Stores in local buffers
2. **Feature Engineering** (`FeatureManager` - lines 1158-1873)
 - Converts raw data into 120+ features

- Features include: connection counts, byte rates, CPU usage, file operations, etc.
- Creates feature vectors every 60 seconds

3. Local Model Training (`train_models()`) - lines 421-619)

- **Isolation Forest**: Trained on feature vectors
- **Autoencoder**: Neural network trained to reconstruct normal patterns
- **OneClassSVM** (optional): Support vector machine for anomaly detection

Training occurs when:

- Initial bootstrap after 3 minutes (if no cached models)
- Periodically every 10 minutes (`ml_training_loop()`) - lines 6150-6186)
- Minimum 20 feature samples required

4. Model Update Preparation (`prepare_fl_update()`) - lines 6025-6141)

```
python

update = {
    'client_id': CLIENT_ID,
    'timestamp': datetime.now(),
    'model_type': 'statistical_weights',
    'weights': detector.model_weights.copy(), # Thresholds & baselines
    'metrics': {
        'samples_used': total_samples,
        'anomaly_rate': anomaly_rate,
        'detection_accuracy': accuracy
    },
    'local_statistics': {
        'network_mean': np.mean(network_data),
        'network_std': np.std(network_data),
        ...
    }
}
```

5. Send to Server (`fl_loop()`) - lines 6187-6230)

- Every 90 seconds
- Sends model weights (thresholds, baselines, sensitivity parameters)
- **Does NOT send model architectures** (Isolation Forest, Autoencoder structures stay local)

Server Side Process:

1. Receive Updates (`handle_fl_update()`) - lines 851-921)

- Stores client updates in queue
- Tracks client reputation based on update quality

2. Aggregation (`aggregate_models()`) - lines 1026-1354)

- **Trigger:** Minimum 2 clients OR manual force
- **Privacy-Enhanced Steps:**

Step 1: Gradient Clipping (Defense against poisoning)

```
python

# Compute delta from global model
delta = local_weights - global_weights

# Calculate L2 norm
delta_norm = sqrt(sum(delta_i^2))

# Clip if too large
if delta_norm > CLIP_BOUND:
    delta = delta * (CLIP_BOUND / delta_norm)
```

Step 2: Trimmed Mean Aggregation (Robust to outliers)

```
python

# For each weight parameter:
values = [client1_weight, client2_weight, ...]

# Sort values
sorted_values = sort(values)

# Remove top/bottom 10%
trimmed = sorted_values[trim_start:trim_end]

# Average remaining
aggregated_weight = mean(trimmed)
```

Step 3: Differential Privacy Noise

```
python

# Add Gaussian noise for privacy
noise = normal(0, DP_NOISE_SCALE)
aggregated_weight += noise
```

Step 4: Validation & Rollback

python

```
# Test new model on validation data
if new_auc < (old_auc - THRESHOLD):
    # Rollback to previous model
    return previous_model
```

3. Convergence Tracking (lines 1261-1279)

python

```
convergence_delta = sqrt(sum((new_weight - old_weight)^2))

convergence_history.append({
    'version': version,
    'delta': convergence_delta,
    'timestamp': now
})
```

4. Broadcast Global Model (`broadcast_global_model()`) - lines 1356-1404)

- Sends updated weights to all clients
- Clients update their local thresholds

2.2 Mathematical Explanation

Convergence Metric: Euclidean Distance

The system tracks convergence using **L2 norm** (Euclidean distance) between successive model versions:

$$\Delta_t = \|\theta_t - \theta_{t-1}\|_2$$

Where:

- θ_t = global model parameters at iteration t
- $\|\cdot\|_2$ = Euclidean norm (sqrt of sum of squares)

Expanded:

If θ has parameters $[w_1, w_2, \dots, w_n]$:

$$\Delta_t = \sqrt{(w_{1_new} - w_{1_old})^2 + (w_{2_new} - w_{2_old})^2 + \dots + (w_{n_new} - w_{n_old})^2}$$

Convergence Criteria:

- **Stable:** $\Delta_t < 0.01$ (model barely changing)
- **Adapting:** $\Delta_t \geq 0.01$ (model still learning)

Why This Metric:

- Captures total parameter change magnitude
- Small delta $\rightarrow$ model has converged to stable point
- Large delta $\rightarrow$ model still adjusting to new client data

Threshold Usage

The threshold delta is used for:

1. **Convergence Monitoring**
 - Displayed in admin GUI
 - Alerts when model is stable vs adapting
2. **Aggregation Frequency Control** (potential use)
 - Could skip aggregation if delta too small
 - Current implementation doesn't use this optimization
3. **Client Reputation** (indirect)
 - Clients causing large deltas might be outliers
 - Affects quality scores in reputation system

3. CURRENT MODELS & TRAINING

3.1 Models in Use

A. Isolation Forest

- **Type:** Ensemble anomaly detection
- **Training:** Unsupervised on client feature vectors
- **Purpose:** Detect outliers by isolation depth
- **Threshold:** Score $< -0.5 \rightarrow$ anomaly
- **Not Shared:** Only statistical weights shared, not tree structures

B. Autoencoder (Neural Network)

python

Input Layer (120 features)



Dense(64) + ReLU + Dropout(0.2)



Dense(32) + ReLU + Dropout(0.2) ← Bottleneck (compressed representation)



Dense(64) + ReLU + Dropout(0.2)



Dense(120) + Sigmoid ← Reconstruction

- **Training:** Learns to reconstruct normal patterns
- **Anomaly Detection:** High reconstruction error = anomaly
- **Threshold:** Set to 90th percentile of training errors
- **Not Shared:** Architecture stays local

C. Statistical Z-Score Filter

- **Formula:** $z = (x - \mu) / \sigma$
- **Threshold:** $|z| > 8.0$
- **Baselines:** EMA (Exponential Moving Average) tracked
- **Shared:** Global mean/std from federated aggregation

D. OneClassSVM (Optional)

- **Type:** Support Vector Machine for novelty detection
- **Kernel:** RBF (Radial Basis Function)
- **Parameter:** $\nu = 0.05$ (5% outlier tolerance)
- **Not Shared:** Model stays local

3.2 Ensemble Scoring

When detecting anomalies, all models vote:

python

```
# Each model provides score 0-10
iso_score = isolation_forest.score(features)
ae_score = autoencoder.reconstruction_error(features)
zscore_score = z_score_filter.score(features)
svm_score = one_class_svm.score(features)

# Weighted average
ensemble_score = (0.3*iso + 0.3*ae + 0.2*z + 0.2*svm)

# Severity levels
if ensemble_score > 7.0: severity = 'critical'
elif ensemble_score > 5.0: severity = 'high'
elif ensemble_score > 3.0: severity = 'medium'
else: severity = 'low'
```

3.3 CRITICAL ISSUE: Training on Own Data

Current Problem:

- Clients train models on **their own live data**
- No validation dataset used
- **Risk:** Models learn site-specific patterns, not generalizable threats
- **Example:** If a client's normal traffic is 100 MB/s, model learns this as "normal"
 - But 100 MB/s might be exfiltration for another client with 10 MB/s baseline

Why This is Wrong:

1. **Overfitting to local environment**
 2. **No ground truth for anomalies**
 3. **Can't validate detection accuracy**
 4. **Federated aggregation combines incompatible baselines**
-

4. RECOMMENDED DATASETS FOR TRAINING

To fix the training issue, models should be pre-trained on **standardized cybersecurity datasets** with labeled anomalies:

4.1 Network Traffic Datasets

A. CICIDS2017 / CICIDS2018

- **Source:** Canadian Institute for Cybersecurity
- **URL:** <https://www.unb.ca/cic/datasets/ids-2017.html>
- **Content:**
 - Benign traffic + 14 attack types
 - DDoS, DoS, brute force, web attacks, infiltration, botnet
- **Size:** ~80 features, 2.8M samples
- **Labels:** Binary (normal/attack) + attack type
- **Use for:** Network anomaly detection models

B. NSL-KDD

- **Source:** Improved version of KDD Cup 99
- **URL:** <https://www.unb.ca/cic/datasets/nsl.html>
- **Content:** Network intrusions with 41 features
- **Attacks:** DoS, Probe, R2L, U2R
- **Use for:** Training Isolation Forest, SVM on network features

C. CTU-13

- **Source:** Czech Technical University
- **URL:** <https://www.stratosphereips.org/datasets-ctu13>
- **Content:** Real botnet traffic captures
- **Labels:** Botnet flows vs normal traffic
- **Use for:** C2 communication detection

D. UNSW-NB15

- **Source:** University of New South Wales
- **URL:** <https://research.unsw.edu.au/projects/unswnb15-dataset>
- **Content:** Modern network traffic with 49 features
- **Attacks:** 9 types including reconnaissance, exploits, worms
- **Use for:** Training ensemble models

4.2 Process/System Behavior Datasets

A. ADFA-LD (Linux)

- **Source:** Australian Defence Force Academy
- **URL:** <https://research.unsw.edu.au/projects/adfa-ids-datasets>
- **Content:** System call traces from Linux servers
- **Attacks:** Privilege escalation, backdoors, data exfiltration
- **Use for:** Process behavior models

B. ADFA-WD (Windows)

- **Same source as above**
- **Content:** Windows event logs and process traces
- **Use for:** Windows client anomaly detection

C. LANL Comprehensive Multi-Source

- **Source:** Los Alamos National Laboratory
- **URL:** <https://csr.lanl.gov/data/cyber1/>
- **Content:** 90 days of process creation, network flows, authentication
- **Size:** 12GB compressed
- **Use for:** Correlating process + network anomalies

4.3 File System Activity Datasets

A. EMBER (Malware Detection)

- **Source:** Elastic Security
- **URL:** <https://github.com/elastic/ember>
- **Content:** 1M+ Windows PE files (benign + malware)
- **Features:** File metadata, imports, sections, byte histograms
- **Use for:** File-based anomaly detection

B. Ransomware Detection Dataset

- **Source:** Various (e.g., VirusTotal, MalwareBazaar)
- **Content:** File I/O patterns from ransomware execution

- **Features:** File access rates, encryption indicators
- **Use for:** Ransomware burst detection model

4.4 User Activity Datasets

A. CMU CERT Insider Threat

- **Source:** Carnegie Mellon University
- **URL:** https://kithub.cmu.edu/articles/dataset/Insider_Threat_Test_Dataset/12841247
- **Content:** Synthetic user behavior with insider threats
- **Features:** Logon/logoff, file access, USB usage, web browsing
- **Use for:** User activity anomaly models

B. WUSTL EHMS

- **Source:** Washington University Enterprise Healthcare Monitoring System
 - **Content:** Real anonymized user activity logs
 - **Use for:** Baseline user behavior modeling
-

5. RECOMMENDED ARCHITECTURE CHANGES

5.1 True Privacy-Preserving Federated Learning

Phase 1: Pre-Training (Offline)

1. Download public datasets (CICIDS2017, ADFA-LD, etc.)
2. Train initial models on labeled data:
 - Isolation Forest on normal vs attack traffic
 - Autoencoder on benign patterns only
 - SVM on labeled anomalies
3. Validate models on test sets (80/20 split)
4. Package pre-trained models for deployment

Phase 2: Client Deployment

Client receives:

- └─ Pre-trained Isolation Forest
- └─ Pre-trained Autoencoder
- └─ Pre-trained SVM
- └─ Global baseline parameters (from federated aggregation)

Client does NOT send raw data to server

Instead:

- └─ Detects anomalies locally using pre-trained models
- └─ Computes local model updates (gradients/weights)
- └─ Sends ONLY model parameters to server
- └─ Optionally: Sends aggregated statistics (counts, means)

Phase 3: Privacy-Enhanced Telemetry

Instead of sending raw data, send:

```
json
{
  "client_id": "abc123",
  "timestamp": "2024-01-10T10:00:00",
  "anomalies_detected": [
    {
      "type": "network",
      "severity": "high",
      "feature_signature": "hash_of_features", // Not raw features
      "model_votes": {"iso": 0.8, "ae": 0.9, "svm": 0.7}
    }
  ],
  "aggregated_stats": {
    "total_connections": 1523,
    "total_processes": 87,
    "anomaly_count": 3
  },
  "model_update": {
    "weight_deltas": [...], // Parameter changes, not raw data
    "update_norm": 0.234
  }
}
```

Admin sees:

- Anomaly counts and types
- Aggregated metrics (no individual connection/process details)

- Model convergence statistics
- Client health status

Admin does NOT see:

- Individual network connections
- Specific process names
- File paths accessed
- User activity details

5.2 Modified Training Pipeline

Client Side:

```
python
```

```

class PrivacyEnhancedDetector:
    def __init__(self):
        # Load pre-trained models
        self.isolation_forest = load_pretrained_model('iso_forest.pkl')
        self.autoencoder = load_pretrained_model('autoencoder.h5')

        # Fine-tune on local data (OPTIONAL)
        self.fine_tune_on_local_data(privacy_budget=0.1)

    def fine_tune_on_local_data(self, privacy_budget):
        """
        Fine-tune with differential privacy
        Only adjusts last layer of autoencoder
        """
        # Add DP noise to gradients
        for gradient in compute_gradients():
            clipped_grad = clip_gradient(gradient, C=1.0)
            noisy_grad = clipped_grad + gaussian_noise( $\sigma$ =privacy_budget)
            apply_update(noisy_grad)

    def prepare_privacy_safe_update(self):
        """
        Compute model update without exposing data
        """
        # Compute gradient differences
        delta_weights = current_weights - previous_weights

        # Clip L2 norm
        norm = l2_norm(delta_weights)
        if norm > CLIP_BOUND:
            delta_weights *= CLIP_BOUND / norm

        # Add DP noise
        delta_weights += gaussian_noise( $\sigma$ =DP_NOISE)

        return {
            'weight_deltas': delta_weights,
            'samples_used': len(local_buffer), # Count only, not data
            'update_norm': norm
        }

```

Server Side:

```
python
```

```

class PrivacyAwareServer:
    def aggregate_updates(self, client_updates):
        """
        Secure aggregation without seeing client data
        """
        # Verify all updates are privacy-compliant
        for update in client_updates:
            assert update['update_norm'] <= CLIP_BOUND
            assert 'raw_features' not in update # Should never be present

        # Trimmed mean aggregation
        aggregated = trimmed_mean_aggregate(client_updates)

        # Add server-side DP noise
        aggregated += gaussian_noise( $\sigma$ =SERVER_DP_NOISE)

        # Validate improvement
        if not self.validate_improvement(aggregated):
            return self.rollback()

        return aggregated

```

5.3 Database Schema Changes

Remove from telemetry table:

```

sql

-- DELETE THESE COLUMNS (privacy violations):
-- network_data (individual connections)
-- process_data (process names, PIDs)
-- file_data (file paths, operations)
-- user_activity (user IDs, login times)

```

Keep only aggregated metrics:

```

sql

```

```

CREATE TABLE telemetry_aggregated (
  id SERIAL PRIMARY KEY,
  client_id VARCHAR(255),
  timestamp TIMESTAMPTZ,

  -- Aggregated counts only
  connection_count INTEGER,
  process_count INTEGER,
  file_operation_count INTEGER,

  -- Anomaly summaries (no raw features)
  anomalies_detected INTEGER,
  high_severity_count INTEGER,

  -- Model metadata
  model_version INTEGER,
  detection_confidence FLOAT
);

CREATE TABLE anomaly_alerts (
  id SERIAL PRIMARY KEY,
  client_id VARCHAR(255),
  timestamp TIMESTAMPTZ,

  -- Anonymized alert info
  severity VARCHAR(20),
  category VARCHAR(50), -- 'network', 'process', 'file'
  feature_signature VARCHAR(255), -- Hash, not actual features

  -- Model scores (not raw data)
  ensemble_score FLOAT,
  model_votes JSONB
);

```

6. FEDERATED LEARNING BEST PRACTICES

6.1 Differential Privacy Parameters

Client-Side:

```
python
```

Per-client privacy budget (ϵ)

`EPSILON_PER_ROUND = 0.1`

Noise scale (σ)

`SIGMA = sqrt(2 * log(1.25/DELTA)) / EPSILON`

Where DELTA = 10^{-5} (failure probability)

Gradient clipping bound

`CLIP_BOUND = 5.0` *# Adjust based on model*

Server-Side:

python

Additional privacy amplification

`SERVER_EPSILON = 0.05`

`SERVER_SIGMA = sqrt(2 * log(1.25/DELTA)) / SERVER_EPSILON`

Total privacy cost after T rounds:

`TOTAL_EPSILON = sqrt(T) * EPSILON_PER_ROUND` *# By composition*

6.2 Secure Aggregation Protocol

For maximum privacy, implement **secure multi-party computation**:

python

Clients create secret shares of their updates

`def create_secret_shares(update, num_clients):`

`shares = []`

`for i in range(num_clients - 1):`

`shares.append(random_vector(size=update.shape))`

Last share makes sum = original update

`last_share = update - sum(shares)`

`shares.append(last_share)`

`return shares`

Server aggregates shares without seeing individual updates

`def aggregate_shares(all_shares):`

`# Each client sends one share to server`

`aggregated = sum(all_shares)` *# Recovers sum of original updates*

`return aggregated / len(all_shares)` *# Average*

6.3 Convergence Monitoring

Better convergence metrics:

```
python

# 1. L2 norm (current implementation)
delta_l2 = ||θ_new - θ_old||_2

# 2. Cosine similarity (direction alignment)
cosine_sim = (θ_new · θ_old) / (||θ_new|| * ||θ_old||)

# 3. Parameter-wise variance
var_params = mean(variance(θ_i across clients))

# 4. Validation AUC improvement
delta_auc = AUC_new - AUC_old

# Convergence criteria:
if (delta_l2 < 0.01) AND (cosine_sim > 0.99) AND (delta_auc < 0.001):
    model_converged = True
```

7. IMPLEMENTATION ROADMAP

Step 1: Immediate Fixes (Week 1)

- 1. Remove raw data transmission from `send_telemetry()`
- 2. Modify database schema to drop sensitive columns
- 3. Update admin GUI to show only aggregated metrics

Step 2: Dataset Integration (Week 2-3)

- 1. Download CICIDS2017, ADFA-LD, NSL-KDD datasets
- 2. Preprocess into feature format matching current FeatureManager
- 3. Train initial models offline:

```
python
```

```
# Train on 80% of dataset
X_train, X_test, y_train, y_test = train_test_split(features, labels, test_size=0.2)

# Isolation Forest
iso_forest.fit(X_train[y_train == 0]) # Fit on normal only

# Autoencoder
autoencoder.fit(X_train[y_train == 0], epochs=50, validation_split=0.2)

# SVM
svm.fit(X_train[y_train == 0])

# Validate
y_pred = ensemble_predict(X_test)
auc = roc_auc_score(y_test, y_pred)
print(f'Validation AUC: {auc}')
```

4. Package models for client deployment

Step 3: Privacy-Enhanced FL (Week 4-6)

1. Implement gradient-only updates
2. Add client-side differential privacy
3. Implement secure aggregation protocol
4. Add server-side DP noise
5. Enhance convergence monitoring

Step 4: Validation & Testing (Week 7-8)

1. Test on holdout datasets
 2. Verify privacy guarantees (no data leakage)
 3. Measure detection accuracy vs centralized baseline
 4. Load testing with 50+ simulated clients
 5. Security audit of FL protocol
-

8. MATHEMATICAL FORMULAS SUMMARY

Differential Privacy Guarantee

For mechanism M and datasets D, D' differing by one record:

$$\Pr[M(D) \in S] \leq e^{\epsilon} \times \Pr[M(D') \in S] + \delta$$

Where:

- ϵ = privacy budget (smaller = more private)
- δ = failure probability (typically 10^{-5})

Gaussian Mechanism:

$$M(D) = f(D) + N(0, \sigma^2)$$

Where $\sigma = (\Delta f / \epsilon) \times \sqrt{2 \times \ln(1.25/\delta)}$

And Δf = sensitivity of function f

Federated Averaging (FedAvg)

Global model update:

$$\theta_{t+1} = \sum (n_k/n) \times \theta_k$$

Where:

- θ_k = local model from client k
- n_k = number of samples at client k
- $n = \sum n_k$ (total samples across all clients)

With gradient clipping:

For each client gradient g_k :

1. Compute norm: $\|g_k\|_2$

2. If $\|g_k\|_2 > C$:

$$g_k \leftarrow g_k \times (C / \|g_k\|_2)$$

3. Aggregate: $g_{\text{global}} = (1/K) \times \sum g_k$

Convergence Delta (L2 Norm)

$$\Delta_t = \|\theta_t - \theta_{t-1}\|_2 = \sqrt{\sum (\theta_t^i - \theta_{t-1}^i)^2}$$

Convergence when: $\Delta_t < \text{threshold}$ (e.g., 0.01)

Ensemble Anomaly Score

$$\text{Score} = w_{\text{iso}} \times S_{\text{iso}} + w_{\text{ae}} \times S_{\text{ae}} + w_{\text{z}} \times S_{\text{z}} + w_{\text{svm}} \times S_{\text{svm}}$$

Where:

- S_{iso} = Isolation Forest score (normalized 0-10)
- S_{ae} = Autoencoder reconstruction error (normalized)
- S_{z} = Z-score magnitude (scaled)
- S_{svm} = SVM decision distance (scaled)
- w_i = weights (sum to 1.0)

9. KEY TAKEAWAYS

Current Problems:

1.  **Privacy violation:** Raw client data sent to server
2.  **No validation datasets:** Models trained on own data
3.  **Overfitting risk:** Site-specific patterns, not generalizable
4.  **Partial FL:** Only statistical weights aggregated, not full models

What Works Well:

1.  **Gradient clipping:** Prevents poisoning attacks
2.  **Trimmed mean:** Robust to outlier clients
3.  **DP noise:** Some privacy protection in aggregation
4.  **Convergence tracking:** Monitors model stability

Recommended Changes:

1.  **Stop sending raw data** - only model parameters + aggregated stats
2.  **Pre-train on public datasets** - ensure models detect real threats
3.  **Implement secure aggregation** - encrypted model updates
4.  **Enhance DP** - client-side and server-side noise
5.  **Validation framework** - use holdout test sets from CICIDS2017 etc.

10. CONCLUSION

Your FortifAI system has a **solid foundation** for federated learning with good privacy mechanisms (clipping, trimming, DP noise) at the server aggregation level. However, it currently **fails to achieve true privacy-preserving FL** because:

1. Raw telemetry data is still transmitted and stored
2. Models are trained on local data without validation datasets
3. No client-side privacy protections

By implementing the recommended changes—especially using public cybersecurity datasets for pre-training and eliminating raw data transmission—you can transform this into a **truly privacy-enhanced federated anomaly detection system** that:

- Protects client privacy (no data leaves client)
- Detects real threats (trained on labeled attack data)
- Scales efficiently (lightweight model updates only)
- Converges reliably (validated on benchmark datasets)

The mathematics behind the FL aggregation is sound, and the convergence delta metric effectively tracks model stability. With the architectural changes outlined above, you'll have a production-ready privacy-preserving cybersecurity system.

APPENDIX: Quick Reference

Public Dataset URLs

- **CICIDS2017:** <https://www.unb.ca/cic/datasets/ids-2017.html>
- **ADFA-IDS:** <https://research.unsw.edu.au/projects/adfa-ids-datasets>
- **NSL-KDD:** <https://www.unb.ca/cic/datasets/nsl.html>
- **UNSW-NB15:** <https://research.unsw.edu.au/projects/unsw-nb15-dataset>
- **EMBER:** <https://github.com/elastic/ember>
- **CMU CERT:** [https://kilthub.cmu.edu/articles/dataset/Insider Threat Test Dataset/12841247](https://kilthub.cmu.edu/articles/dataset/Insider_Threat_Test_Dataset/12841247)

Privacy Parameters

```
python
```

EPSILON_CLIENT = 0.1 # Client privacy budget per round
DELTA = 1e-5 # Failure probability
CLIP_BOUND = 5.0 # Gradient L2 norm bound
TRIM_FRACTION = 0.1 # Trim top/bottom 10%
DP_NOISE_SCALE = 0.5 # Noise multiplier
CONVERGENCE_THRESHOLD = 0.01 # Model stability threshold

Feature Counts by Category

- **Network:** 35 features (connection counts, byte rates, protocols, etc.)
- **Process:** 28 features (CPU, memory, thread counts, etc.)
- **File:** 32 features (I/O rates, file types, access patterns)
- **User:** 25 features (login times, activity levels, privilege changes)
- **Total:** 120+ features

Document Version: 1.0
Last Updated: 2026-01-10
Author: System Architecture Analysis